

ECE 528/L

Robotics and Embedded Systems with Lab

Lab 0

General Purpose Input Output (GPIO)

Author: Aaron Nanas
Fall 2026

Table of Contents

1. Introduction.....	2
2. Objectives.....	2
3. Required Components.....	2
4. Pre-Lab.....	3
4.1 Background.....	3
4.2 Register Configuration.....	5
4.2.1 Output Initialization	5
4.2.2 Input Initialization.....	7
5. Procedure.....	9
6. Tasks	20
7. Resources	26

1. Introduction

This lab focuses on the General-Purpose Input/Output (GPIO) interface of the MSP432 LaunchPad and introduces basic functionality such as reading input from the two switches and controlling the two LED outputs. In addition, the values of the memory-mapped registers of the Input/Output (I/O) ports will be observed.

2. Objectives

In this lab, you will learn the following:

1. Review fundamentals of C programming
2. Configuring the I/O ports on the MSP432 LaunchPad
3. Interacting with the built-in user buttons and LEDs on the MSP432 LaunchPad
4. Interfacing external modules such as the PMOD 8LD and PMOD SWT by connecting them to the I/O ports
5. Creating LED patterns based on the statuses of the buttons and the switches
6. Analyzing the I/O ports by viewing the memory-mapped registers

3. Required Components

This lab uses the MSP432 LaunchPad with some hardware required. The preferred operating system to use for this course is Windows.

Description	Quantity	Manufacturer
MSP432 LaunchPad	1	Texas Instruments
USB-A to Micro-USB Cable	1	N/A
PMOD 8LD	1	Digilent
PMOD SWT	1	Digilent

4. Pre-Lab

4.1 Background

Figure 4.1-1 illustrates the GPIO block diagram that consists of the components used in the lab. There are built-in user LEDs and buttons located on the MSP432 LaunchPad. Both the PMOD 8LD and PMOD SWT are connected using the header pins of the MSP432 LaunchPad. In addition, a 48 MHz clock is used.

Note: Pins are not 5V tolerant, so all inputs to the pins must be within the range of 0V to 3.3V.

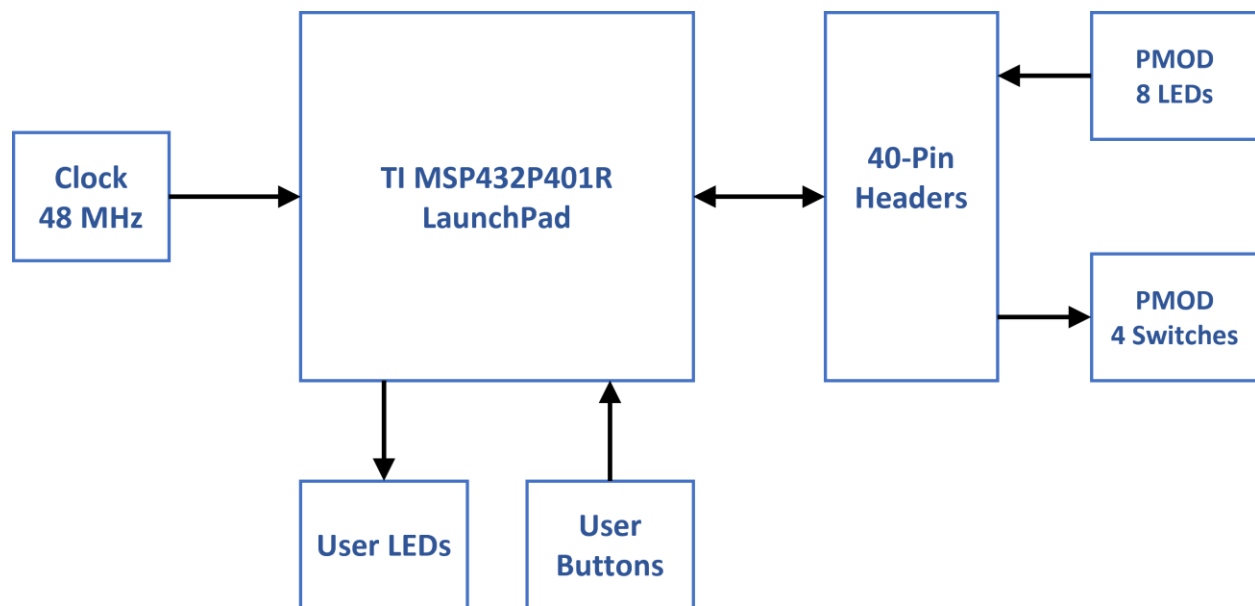


Figure 4.1-1: GPIO Block Diagram

When the two user buttons are pressed, they connect to the GND of the MSP432 LaunchPad, as shown in the schematic in Figure 4.1-2, providing a value of 0V. The first button, labeled as S1, is located at pin P1.1, while the second button, labeled as S2, is located at pin P1.4.

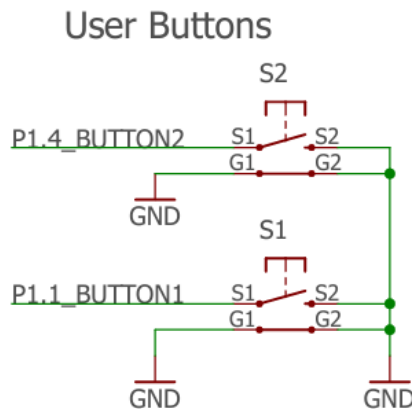


Figure 4.1-2: User Buttons Schematic

Additionally, as shown in the schematic in Figure 4.1-3, the user LEDs consist of a red LED located at P1.0 and RGB LEDs located at pins P2.0, P2.1, and P2.2.

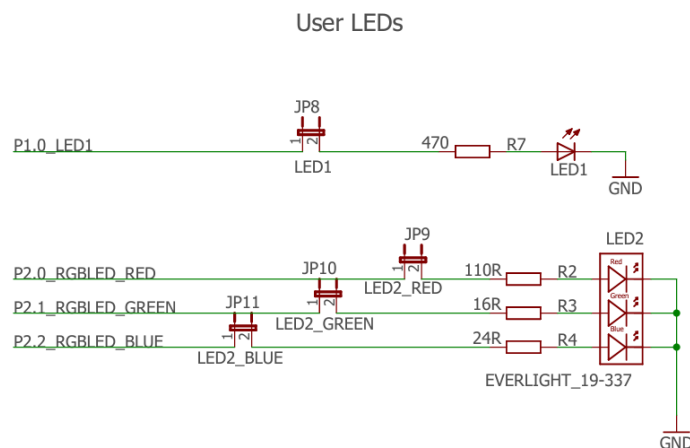


Figure 4.1-3: User LEDs Schematic

4.2 Register Configuration

The pins of the MSP432 LaunchPad can be configured by modifying the values of the memory-mapped registers. A digital I/O pin can be configured either as an input or an output.

4.2.1 Output Initialization

As an example, the P1.0 pin can be configured as a GPIO output pin by following the configuration steps.

The line of code below performs the following:

- **P1->SEL0** refers to the SEL0 register of Port 1 (P1) of the MSP432 LaunchPad. The operator **->** is used to access a member of a structure using a pointer
- A bitwise AND assignment operation (**&=**) is performed between **P1->SEL0** and the complement of **0x01** (in this case, the value is 0xFE) and stores the new value in **P1->SEL0**

The purpose of this line is to clear the least significant bit (LSB) of the **SEL0** register in Port 1 (P1). The **SEL0** register is responsible for selecting the peripheral function of a pin. Clearing the LSB of Port 1 ensures that the pin is configured for general-purpose I/O (GPIO) mode.

```
P1->SEL0 &= ~0x01;
```

The same configuration steps must be applied to the **SEL1** register to ensure that the pin is configured for GPIO mode. Note that the bit positions correspond to the pins. The first bit refers to the P1.0 pin. For example, if the P1.1 pin were selected instead of P1.0, then the second bit would be modified in this case, changing the value to 0x02.

```
P1->SEL1 &= ~0x01;
```

Lastly, the line of code below performs the following to set the direction of the pin:

- **P1->DIR** refers to the direction (DIR) register of Port 1 of the MSP432 Launchpad
- A bitwise OR assignment operation (**|=**) is performed between **P1->DIR** and **0x01** and stores the new value in **P1->DIR**

The purpose of this line is to set the LSB of **P1->DIR** to 1. The DIR register is responsible for setting the direction of a pin. The LSB is set to 1 while leaving the other bits unaffected, configuring the pin as an output.

```
P1->DIR |= 0x01;
```

Combining the three lines of code will initialize pin P1.0, which is LED1 as shown in Figure 4.1-3. In the **GPIO.c** file, LED1 is initialized in the **LED1_Init** function.

```
void LED1_Init()
{
    P1->SEL0 &= ~0x01;
    P1->SEL1 &= ~0x01;
    P1->DIR |= 0x01;
}
```

4.2.2 Input Initialization

Additionally, the following configuration steps provide details on how to configure the P1.1 and P1.4 pins as GPIO input pins.

The line of code below performs the following:

- **P1->SEL0** refers to the SEL0 register of Port 1 of the MSP432 Launchpad
- A bitwise AND assignment operation (&=) is performed between **P1->SEL0** and the complement of **0x12** (in this case, the value is 0xED) and stores the new value in **P1->SEL0**

The purpose of this line is to clear the bits corresponding to the P1.1 and P1.4 pins and configure them as GPIO pins.

```
P1->SEL0 &= ~0x12;
```

The same configuration steps must be applied to the **SEL1** register to ensure that the pin is configured for GPIO mode.

```
P1->SEL1 &= ~0x12;
```

Lastly, the line of code below performs the following to set the direction of the pin:

- **P1->DIR** refers to the direction (DIR) register of Port 1 of the MSP432 Launchpad
- A bitwise AND assignment operation (&=) is performed between **P1->DIR** and the complement of **0x12** and stores the new value in **P1->DIR**

The purpose of this line is to set the direction of P1.1 and P1.4 as input pins by setting the corresponding bits to 0.

```
P1->DIR &= ~0x12;
```

To ensure that the P1.1 and P1.4 pins are in a known state when no external signal is connected, the internal resistors can be enabled. The lines of code below perform the following:

- **P1->REN** refers to the Resistor Enable (REN) register of Port 1 of the MSP432 LaunchPad
- A bitwise OR assignment operator (**|=**) is performed between **P1->REN** and **0x12** and stores the result in **P1->REN**. This enables the internal resistors to ensure that the input pins are in a known state.
- **P1->OUT** refers to the Output register of Port 1 of the MSP432 LaunchPad which controls the output state of the pin
- A bitwise OR assignment operator (**|=**) is performed between **P1->OUT** and **0x12** and stores the result in **P1->OUT**. Since the user buttons are active low, this ensures that the initial output state of P1.1 and P1.4 are logic HIGH, indicating that pull-up resistors are active.

```
P1->REN |= 0x12;
```

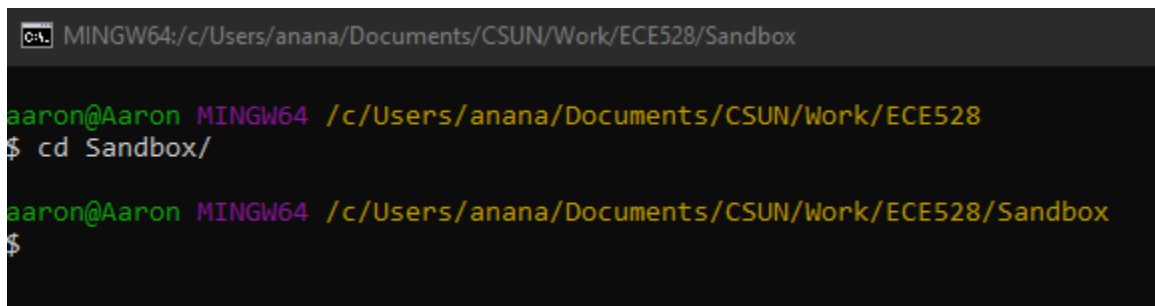
```
P1->OUT |= 0x12;
```

In the **GPIO.c** file, the user buttons are initialized in the **Buttons_Init** function.

```
void Buttons_Init()
{
    P1->SEL0 &= ~0x12;
    P1->SEL1 &= ~0x12;
    P1->DIR &= ~0x12;
    P1->REN |= 0x12;
    P1->OUT |= 0x12;
}
```

5. Procedure

1. Open Git Bash and change the current directory to your workspace directory.

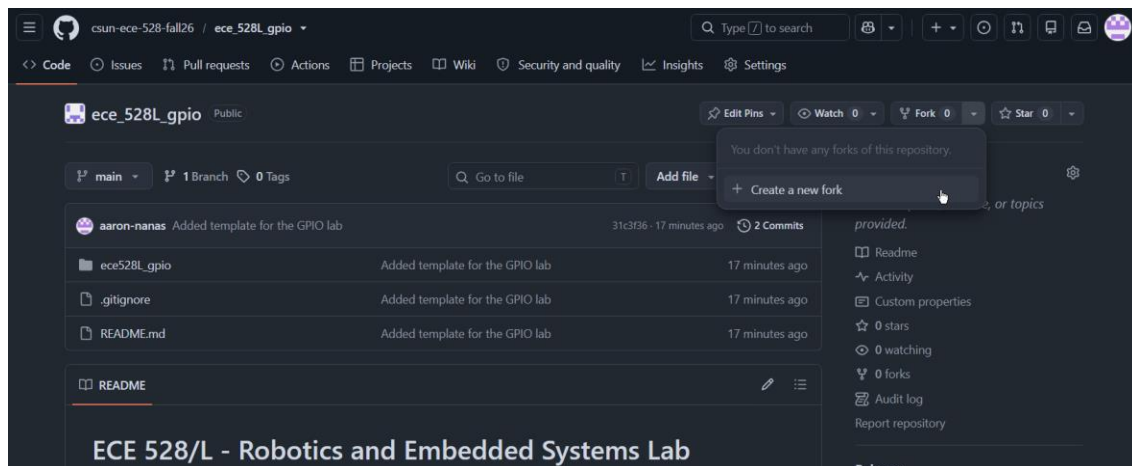


```
MINGW64:/c:/Users/anana/Documents/CSUN/Work/ECE528/Sandbox

aaron@Aaron MINGW64 /c:/Users/anana/Documents/CSUN/Work/ECE528
$ cd Sandbox/

aaron@Aaron MINGW64 /c:/Users/anana/Documents/CSUN/Work/ECE528/Sandbox
$
```

2. Fork the [GPIO lab repository](#) and clone your forked repository via SSH. Import the Code Composer Studio project folder after cloning the repository.



3. Connect the PMOD SWT to the MSP432 LaunchPad using the following pinout. Visit the [PMOD SWT product page](#) for more details regarding the pinout.

Note: Always make sure that both the MSP432 LaunchPad and the motor board are OFF before making connections. Double-check your connections after making them.

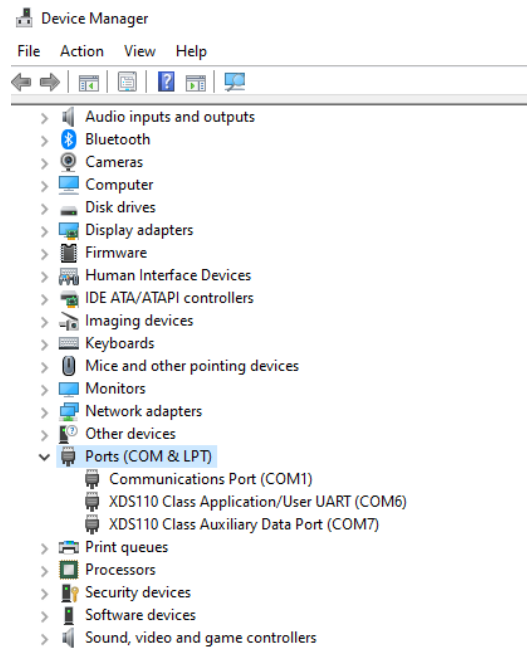
PMOD SWT Pin	MSP432 LaunchPad Pin
SWT1	P10.0
SWT2	P10.1
SWT3	P10.2
SWT4	P10.3
Pin 5 (GND)	GND
Pin 6 (VCC)	VCC (3.3V)

4. Connect the PMOD 8LD to the MSP432 LaunchPad using the following pinout. Visit the [PMOD 8LD product page](#) for more details regarding the pinout.

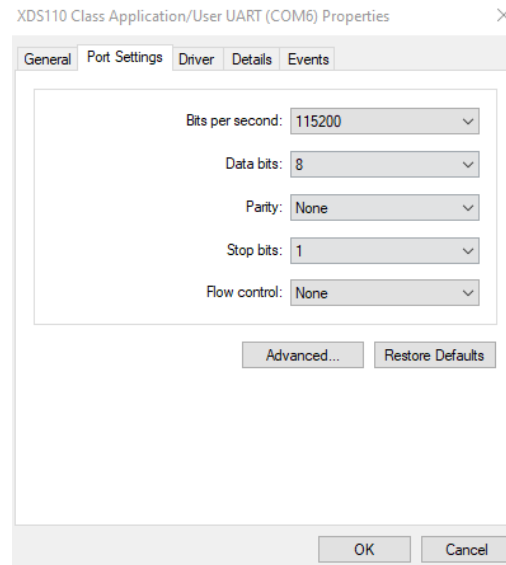
Alternatively, instead of using a PMOD 8LD module, you can use two PMOD LED modules—each one has four LEDs. Refer to the [PMOD LED product page](#) if you are using one.

PMOD 8LD Pin	MSP432 LaunchPad Pin
LED0	P9.0
LED1	P9.1
LED2	P9.2
LED3	P9.3
Pin 5 (GND)	GND
Pin 6 (VCC)	VCC (3.3V)
LED4	P9.4
LED5	P9.5
LED6	P9.6
LED7	P9.7
Pin 11 (GND)	GND
Pin 12 (VCC)	VCC (3.3V)

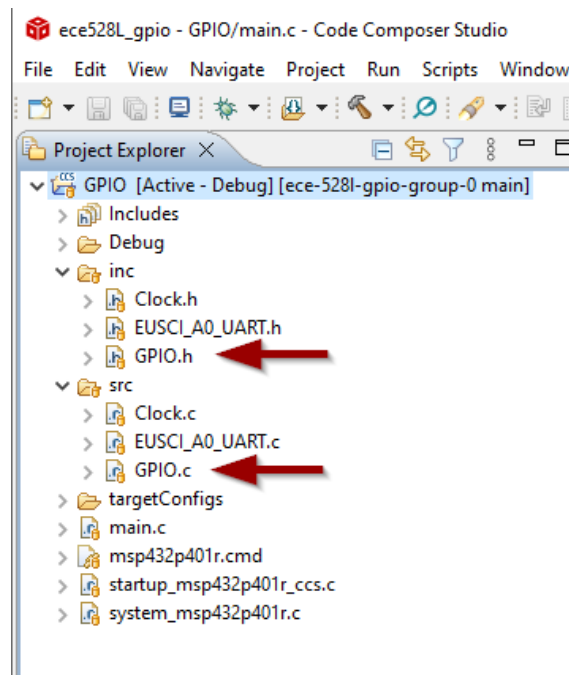
5. Verify your PMOD connections with the instructor. Then, connect the MSP432 LaunchPad using a USB-A to Micro-USB Cable to your computer. Note that two COM ports will appear in Device Manager. For this lab, we will be focusing on using the **Application/User UART COM** port.



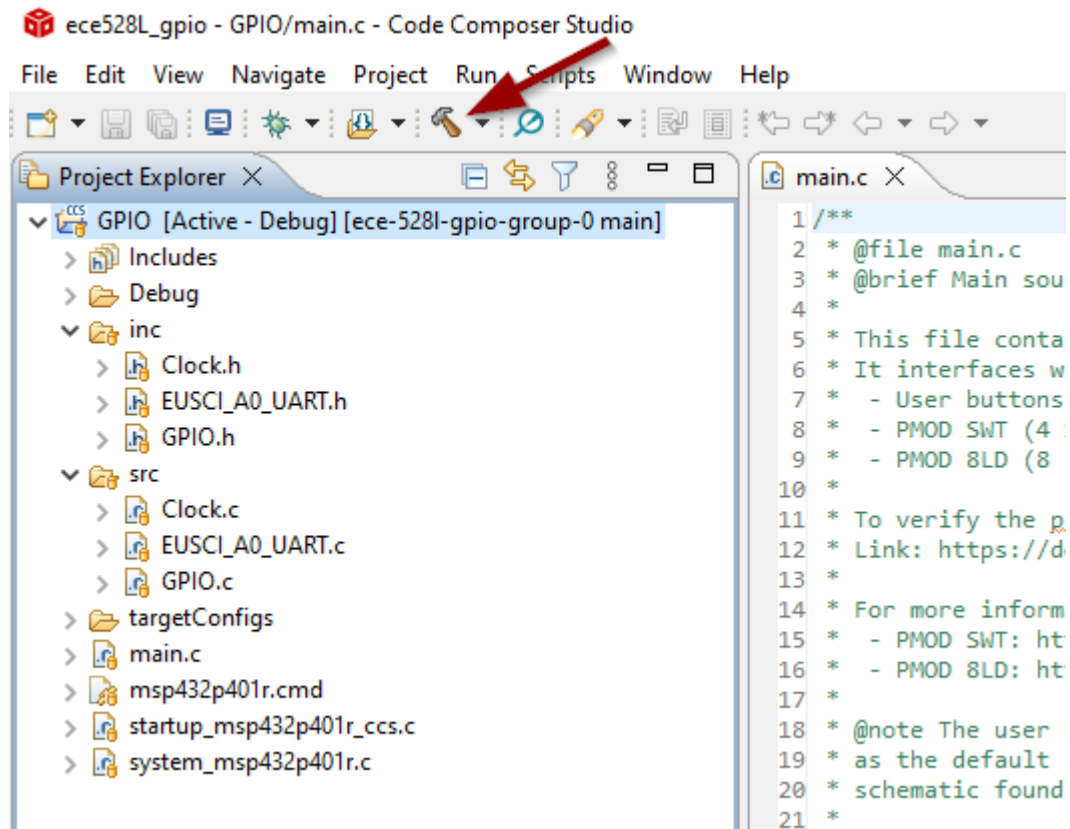
- Verify that the **Application/User UART COM** port has the correct port settings as shown in the figure below.



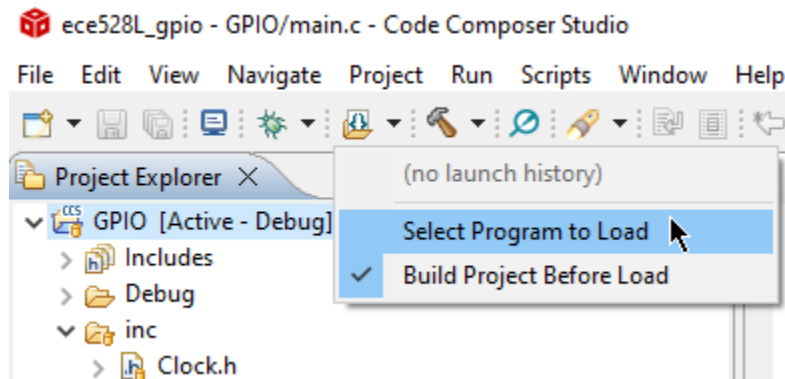
- In the **Project Explorer** window of Code Composer Studio, expand the **GPIO** directory. Review the **GPIO.c**, **GPIO.h**, and **main.c** files and read the code documentation.



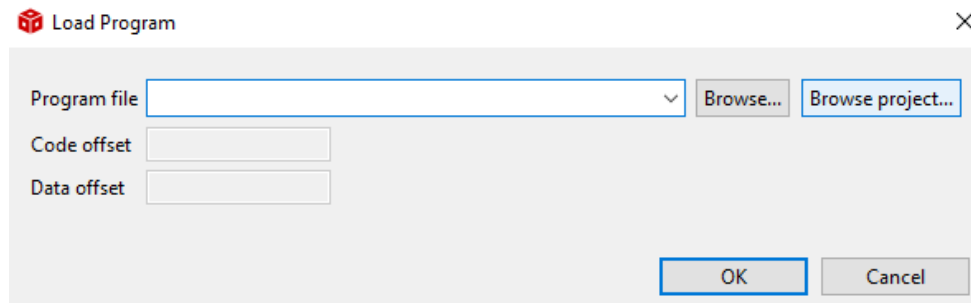
8. Double-click on the **main.c** file to view the main program. Build the project by clicking the hammer icon as shown below.



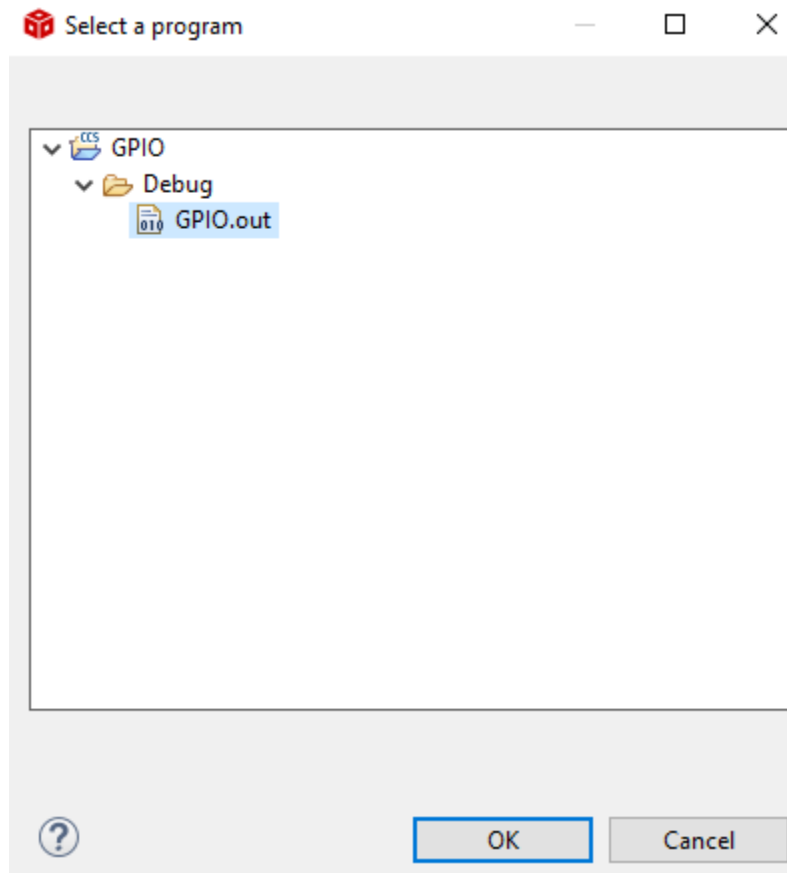
9. After the **GPIO** project has been built, it must be loaded before flashing the program to the MSP432 LaunchPad.



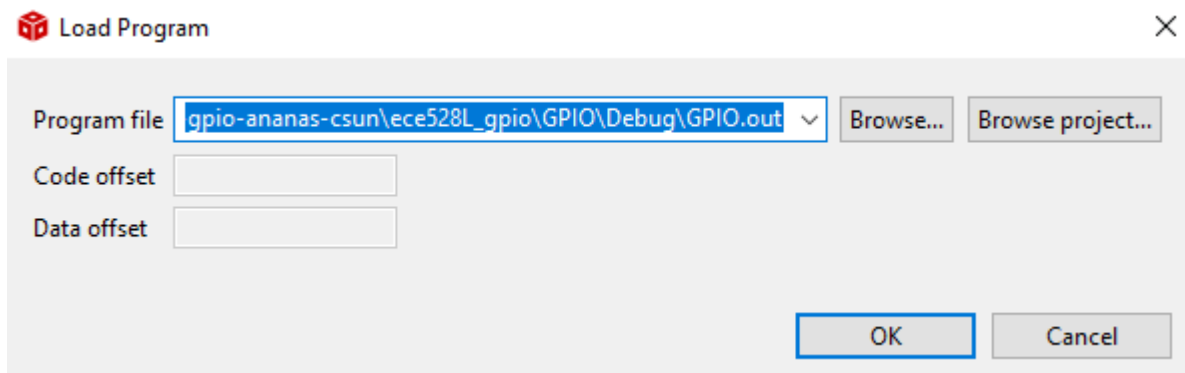
10. Select **Browse project...** and look for the **GPIO** project.



11. Then, expand **GPIO**, select the **GPIO.out** file, and hit OK.



12. Verify that **GPIO.out** has been selected and hit OK to load the program.



13. After programming the board, verify the following I/O behavior.

When all of the switches on the PMOD SWT are off, pressing the buttons will affect the LEDs.

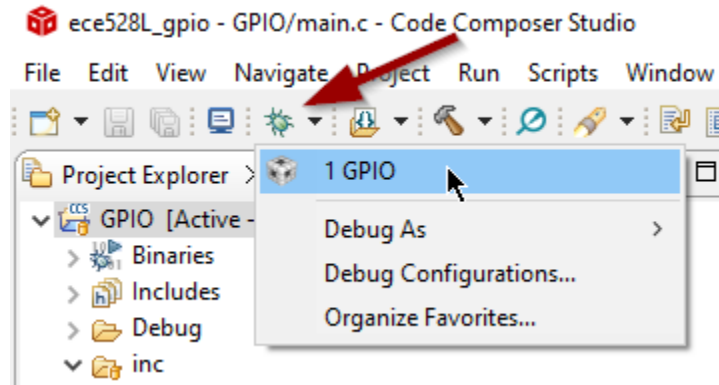
Test Case	Input	LED 1	RGB LED	PMOD 8LD
0	Button 1 is pressed	ON	OFF	LEDs 0 – 3: ON LEDs 4 – 7: OFF
1	Button 2 is pressed	OFF	GREEN	LEDs 0 – 3: OFF LEDs 4 – 7: ON
2	Both Button 1 and Button 2 are pressed	ON	GREEN	LEDs 0 – 7: ON
3	Neither buttons are pressed	OFF	OFF	LEDs 0 – 7: OFF

When only SWT1 is enabled, pressing the buttons will not affect the LEDs. Both the LED 1 and the RGB LED will display a red color, while the LEDs on the PMOD 8LD module will light up in a pattern where each LED corresponds to a specific bit of a binary value. Starting from 0, the LEDs will increment by 1 in binary representation until the maximum value of 255 is reached. This pattern represents an 8-bit binary up counter.

Test Case	Input	LED 1	RGB LED	PMOD 8LD
0	Only SWT1 is enabled	ON	RED	Binary Up Counter

Note: Verify that you've completed this step with the instructor.

14. Initiate a debug session to observe the values of the memory-mapped registers.



15. Expand the **P1** register on the **Registers** window. Click **Step Over** until after the **LED1_Init** function has been called. The value of the **P1DIR** register should be updated. Record the value of **P1DIR** and save a screenshot of the entire CCS window. The screenshot file name should follow the syntax:

- ece528L_lab0_gpio_port1.png

16. Expand the **P2** register on the **Registers** window. Click **Step Over** until after the **LED2_Init** function has been called. The values of the **P2DIR** and **P2DS** registers should be updated. Record these values and save a screenshot of the entire CCS window. The screenshot file name should follow the syntax:

- ece528L_lab0_gpio_port2.png

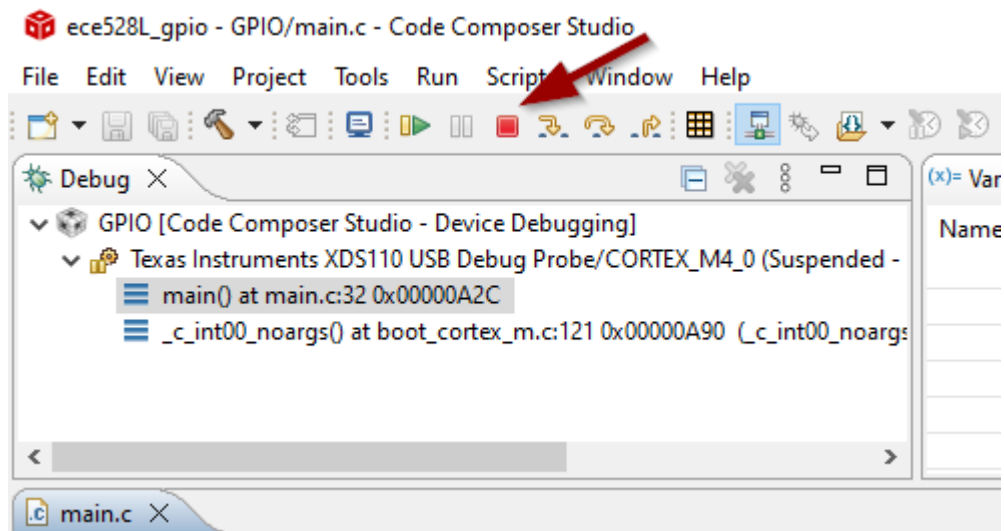
17. Expand the **P9** register on the **Registers** window. Click **Step Over** until after the **PMOD_8LD_Init** function has been called. The value of the **P9DIR** register should be updated. Record the value of **P9DIR** and save a screenshot of the entire CCS window. The screenshot file name should follow the syntax:

- ece528L_lab0_gpio_port9.png

18. Expand the **P10** register on the **Registers** window. Click **Step Over** until after the **PMOD_SWT_Init** function has been called. The value of the **P10DIR** register should remain unchanged. Record the value of **P10DIR** and save a screenshot of the entire CCS window. The screenshot file name should follow the syntax:

- ece528L_lab0_gpio_port10.png

19. Click the **Terminate** button to exit out of the debugger and terminate execution. Make sure to save your screenshots in your repository. It is recommended that you create a separate folder for screenshots.



6. Tasks

1. Modify the *LED_Pattern_1* function to match the following test cases. Then, update the code documentation in the **GPIO.h** header file after modifying the function.

Hint: Use the *Clock_Delay1ms* function to implement time delays.

Test Case	Input	LED 1	RGB LED	PMOD 8LD
0	Button 1 is pressed	ON	OFF	LEDs 0, 2, 4, 6: ON LEDs 1, 3, 5, 7: OFF
1	Button 2 is pressed	OFF	BLUE	LEDs 0, 2, 4, 6: OFF LEDs 1, 3, 5, 7: ON
2	Both Button 1 and Button 2 are pressed	Toggle every 1 second	GREEN, toggle every 1 second	LEDs 0 – 7: OFF
3	Neither buttons are pressed	OFF	OFF	LEDs 0 – 7: ON

2. Create a *void* function named *LED_Pattern_3* that takes no arguments. This function should turn on LED 1, display a blue color on the RGB LED, and output an 8-bit binary down counter pattern on the PMOD 8LD module that counts down from 255 to 0. The delay rate for each count should be 100 ms.

This function should be called only when SWT2 is enabled. The sequence should stop when other switches are enabled. Since SWT2 is being used, another case statement in the *LED_Controller* function will need to be added. Include code documentation for the function in the **GPIO.h** header file.

Test Case	Input	LED 1	RGB LED	PMOD 8LD
0	Only SWT2 is enabled	ON	BLUE	Binary Down Counter

3. Create a *void* function named *LED_Pattern_4* that takes no arguments. This function should generate a ring counter pattern by setting one bit every iteration. In the first iteration, the least significant bit is set to 1. Then, for every iteration, the “1” bit shifts to the left, rotating through every bit position until it returns to the original position. The delay rate for each iteration should be 200 ms.

This function should be called only when SWT3 is enabled. The sequence should stop when other switches are enabled. Since SWT3 is being used, another case statement in the *LED_Controller* function will need to be added. Include code documentation for the function in the **GPIO.h** header file.

Iteration	LED 1	RGB LED	PMOD 8LD
0	OFF	OFF	1
1	OFF	OFF	2
2	OFF	OFF	4
3	OFF	OFF	8
4	OFF	OFF	16
5	OFF	OFF	32
6	OFF	OFF	64
7	OFF	OFF	128

4. Create a *void* function named *LED_Pattern_5* that takes no arguments. This function should generate a ring counter pattern by setting one bit every iteration. In the first iteration, the most significant bit is set to 1. Then, for every iteration, the “1” bit shifts to the right, rotating through every bit position until it returns to the original position. The delay rate for each iteration should be 200 ms.

This function should be called only when SWT4 is enabled. The sequence should stop when other switches are enabled. Since SWT4 is being used, another case statement in the *LED_Controller* function will need to be added. Include code documentation for the function in the **GPIO.h** header file.

Iteration	LED 1	RGB LED	PMOD 8LD
0	OFF	OFF	128
1	OFF	OFF	64
2	OFF	OFF	32
3	OFF	OFF	16
4	OFF	OFF	8
5	OFF	OFF	4
6	OFF	OFF	2
7	OFF	OFF	1

5. Create a *void* function named *Johnson_Counter* that takes no arguments. This function should generate an 8-bit Johnson (also known as twisted ring) counter pattern. It should start the sequence at all zeroes and update the pattern by shifting left by one bit and inserting the inverted previous MSB into Bit 0. An example sequence is shown in the table below. The delay rate for each iteration should be 200 ms.

This function should be called only when SWT0 and SWT1 are enabled, while all other switches are disabled. The sequence should stop when other switches are enabled. Since SWT0 and SWT1 are being used, another case statement in the *LED_Controller* function will need to be added. Include code documentation for the function in the **GPIO.h** header file.

Iteration	LED 1	RGB LED	PMOD 8LD (Binary)
0	ON	GREEN	0000_0000
1	ON	GREEN	0000_0001
2	ON	GREEN	0000_0011
3	ON	GREEN	0000_0111
4	ON	GREEN	0000_1111
5	ON	GREEN	0001_1111
6	ON	GREEN	0011_1111
7	ON	GREEN	0111_1111
8	ON	GREEN	1111_1111
9	ON	GREEN	1111_1110
10	ON	GREEN	1111_1100

Iteration	LED 1	RGB LED	PMOD 8LD (Binary)
11	ON	GREEN	1111_1000
12	ON	GREEN	1111_0000
13	ON	GREEN	1110_0000
14	ON	GREEN	1100_0000
15	ON	GREEN	1000_0000
0 (repeat)	ON	GREEN	0000_0000 (repeat)

7. Resources

1. MSP432P401R SimpleLink Microcontroller LaunchPad Development Kit User's Guide
 - [PDF Link](#)
2. MSP432P401R Datasheet
 - [PDF Link](#)
3. Robot Systems Learning Kit (TI-RSLK) User Guide
 - [PDF Link](#)
4. MSP432P4xx SimpleLink Microcontrollers Technical Reference Manual
 - [PDF Link](#)
5. PMOD SWT Reference Manual
 - [Reference Manual Link](#)
6. PMOD LED Reference Manual
 - [Reference Manual Link](#)
7. PMOD 8LD Reference Manual
 - [Reference Manual Link](#)